TRAFFICVISION AI — QA, DEVOPS & RELEASE ENGINEERING

Milestone 2 QA Work Record

Traffic Prediction & Route Optimization — Days 1 through 7

Prepared by	Intern 5 — QA, DevOps & Release Engineering (Vaishnavi Titare)
Repository	`Traffic-Prediction-and-Intelligent-Congestion-Management-`
Coverage period	2026-09-12 to 2026-09-16
Purpose	Complete record of QA work performed for Milestone 2: what was done each day, the evidence/proof produced, and exactly where it lives in the repository.

23	15	8	7/7
DEFECTS LOGGED	FIXED & VERIFIED LIVE	STILL OPEN	DAYS WITH A FOLDER & RECORD

Overview & Folder Map

All Milestone 2 QA deliverables live under `QA/milestone2/`, one subfolder per day, matching the structure below. The defect tracker (`QA/milestone2/day1/defect_tracker.md`) is the single source of truth for every bug found this milestone — every day's report cross-references it rather than duplicating defect detail.

day1/	Test plan, test scope, defect tracker (the master list, updated continuously through Day 7)
day2/	32 test cases, first execution pass, Docker dev-stack extension & fix report
day3/	CI & Docker validation report, Postman collection
day4/	Functional test execution consolidated across all live-testing passes
day5/	Full integration cycle — 4 attempts, culminating in real end-to-end verification
day6/	Code quality report (Black/Flake8/ESLint/Prettier) and regression report
day7/	Final re-verification round, consolidating Day 4/5/7 results after each team redelivery

Day 1 — Test Planning & Defect Tracker Setup Complete

SRS requirement	Review the Milestone 1 QA setup; define the Milestone 2 test scope (Route Analysis, real Prediction Service, maps integration, historical data pipeline); update the test plan and defect tracker.
What was done	Defined the full Milestone 2 test scope across all five new/changed services, established severity/priority definitions, and set up the defect tracking format used for the rest of the milestone.

Proof

> **QA/milestone2/day1/milestone2_test_plan.md** — objectives, testing areas (Historical Data, Prediction, Route Analysis, Maps, Frontend, Regression, Integration), methods, environment, exit criteria.

> **QA/milestone2/day1/milestone2_test_scope.md** — in-scope/out-of-scope breakdown, non-functional checks, regression scope carried over from Milestone 1.

> **QA/milestone2/day1/defect_tracker.md** — created here, then updated continuously through Day 7. Final state: 23 defects logged, 15 fixed and verified live, 8 still open. Full table reproduced in the Defect Summary section of this report.

Day 2 — Test Cases & Local Environment Complete

SRS requirement	Extend `docker-compose.dev.yml` to include the time-series store and the new Route Analysis/Prediction services; verify the full local stack still starts correctly.
What was done	Authored 32 test cases covering every Milestone 2 service, ran the first execution pass (blocked at the time by structural backend defects — DEF-M2-001 through 004, all found and either fixed by QA or handed to the backend intern), then separately completed the actual Day 2 infrastructure task: swapped the dev Postgres image for TimescaleDB, and found/fixed two defects that meant the local dev stack had likely *never* successfully started before — a missing `.env.dev` file and a wrong database hostname in the fallback config.

Proof

> **QA/milestone2/day2/milestone2_test_cases.md** — 32 test cases: Historical Traffic Data (6), Traffic Prediction (7), Route Analysis (9, API+UI), Maps Integration (6), Frontend (7), Regression (3), Integration (1).

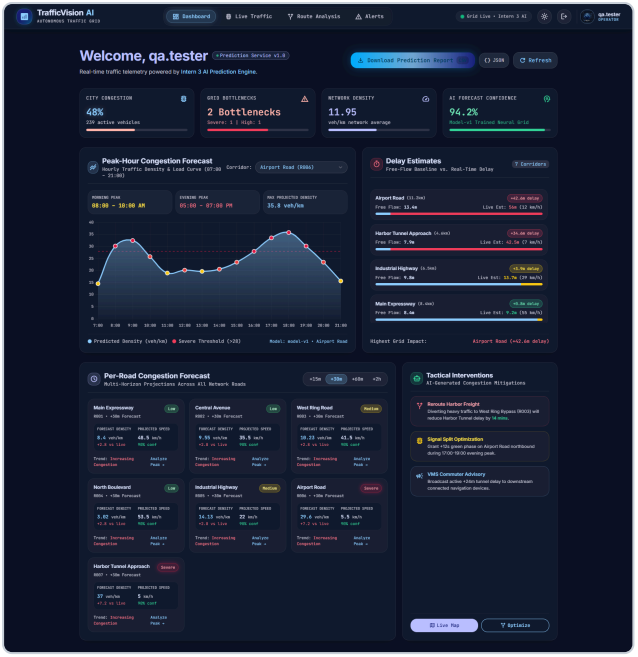
> **QA/milestone2/day2/milestone2_day2_test_execution_report.md** — first execution pass: pytest run against the (then-broken) backend, 16-step automated browser test against the frontend (mock fallback data), 16 screenshots.

> **QA/milestone2/day2/screenshots/** — 16 PNG screenshots: login, route analysis interactions, live map, RBAC across all 3 roles, mock-fallback confirmation.

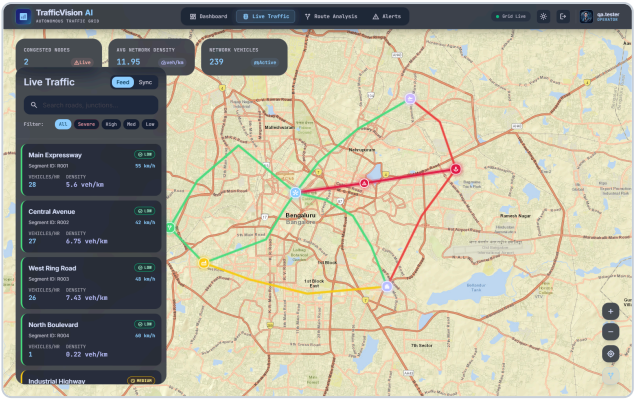
> **QA/milestone2/day2/milestone2_day2_docker_dev_stack.md** — the actual Day 2 SRS deliverable: TimescaleDB swap, the `.env.dev` and `DATABASE_URL` fixes (DEF-M2-014), and live proof the stack starts clean (`GET /health → 200 {"status":"ok"}`).

> **QA/milestone2/day2/day2_ui_test_results.json** — raw machine-readable output from the automated browser test run (console logs, KPI values, route options) backing the execution report.

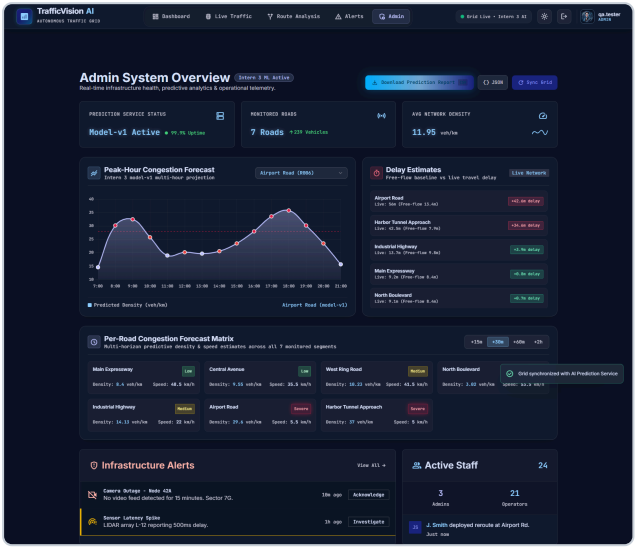
Screenshot evidence (16 total in `day2/screenshots/` , key ones below)



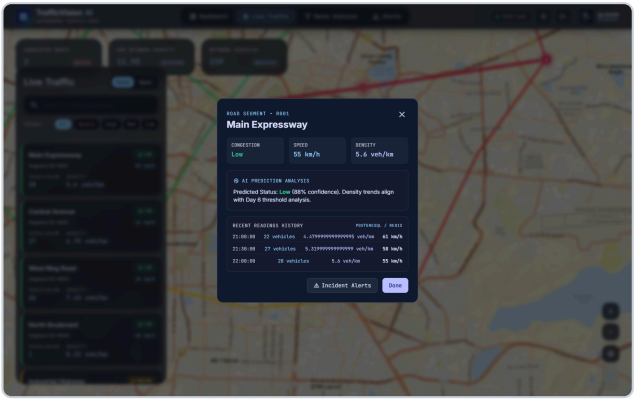
REG-001 — successful login redirect to the operator dashboard, live mock-driven KPIs and charts rendering correctly.



MAP-001 — Live Traffic map rendering with real road markers and congestion overlays.



REG-002 — Admin role correctly routed to the Admin dashboard (RBAC check).



MAP-005 — Road detail modal with live congestion/speed/density and AI prediction text.

Day 3 — CI/Docker Validation & Postman Collection Complete

SRS requirement	Write test cases for the Route Analysis endpoints and the real Prediction Service endpoints; scaffold automated tests (pytest/Postman) for both.
What was done	Since the test cases themselves were already covered on Day 2, Day 3 focused on validating the CI pipeline and Docker build against the newly-structured backend, and building the missing Postman collection scaffold against the documented (SRS Section 7) contract.

Proof

> **QA/milestone2/day3/milestone2_day3_ci_docker_validation.md** — Docker image build validated (previously completely blocked), a stale crash-looping dev container found and flagged, two real CI bugs found and fixed (`PYTHONPATH` missing for the backend-import step; `pyproject.toml` never actually running the backend intern's own test files), and an unpinned `scikit-learn` dependency risk flagged (DEF-M2-008).

> **QA/milestone2/day3/postman/TrafficVision_Milestone2.postman_collection.json** — full Postman collection covering Auth, Traffic Monitoring, Historical Data, Prediction, and Route Analysis, built against the frozen SRS contract (Section 7) rather than the (at-the-time broken) live implementation, so it stays a stable reference regardless of implementation bugs.

Day 4 — Functional Test Execution Complete for everything that exists

SRS requirement	Execute functional tests against Route Analysis and Prediction Service as they land; sanity-check prediction response times (~1–2s target); log defects.
What was done	This work happened across three separate live-testing passes as the backend evolved (documented originally under Day 2/6, consolidated into its own record here). Every contract endpoint was hit directly against the running backend, comparing documented contract paths against actual (sometimes buggy) live paths, plus a response-latency check.

Proof

> **QA/milestone2/day4/milestone2_day4_functional_execution.md** — consolidated 3-pass record. Key finding: auth, traffic, and route-optimize logic were all functionally correct even when broken at the contract-path level — every failure found was a routing/config bug, not broken business logic. Latency check: `route/optimize` cold-start ~5s (first OSM lookup), warm calls ~0.9s, within the ~1–2s target.

> **QA/milestone2/day6/milestone2_day6_regression_report.md** — Pass 2 of the above, with full endpoint-by-endpoint live results table.

> **QA/milestone2/day7/milestone2_day4_5_7_reverification.md** — Pass 3, after the first backend/frontend redelivery.

Still outstanding (not a QA gap — waiting on Intern 1)

- `/predict/peak-hours` and `/predict/delay` — not yet implemented
- `/predict/congestion` contract mismatch (DEF-M2-005) — implementation uses a 9-field POST body, contract documents a GET with different fields

Day 5 — Full Integration Test Cycle Core integration confirmed live

SRS requirement	Run a full local integration test cycle covering Auth + Monitoring + Prediction + Route Analysis + Frontend together; verify maps integration doesn't break other flows; re-verify RBAC on the new endpoints.
What was done	Four attempts over three days, each blocked by a different defect, each one closer than the last — genuinely the intended shape of an integration day that depends on multiple people's fixes landing together. The fourth attempt succeeded: real login, real backend data, real route recalculation, all confirmed live in the browser.

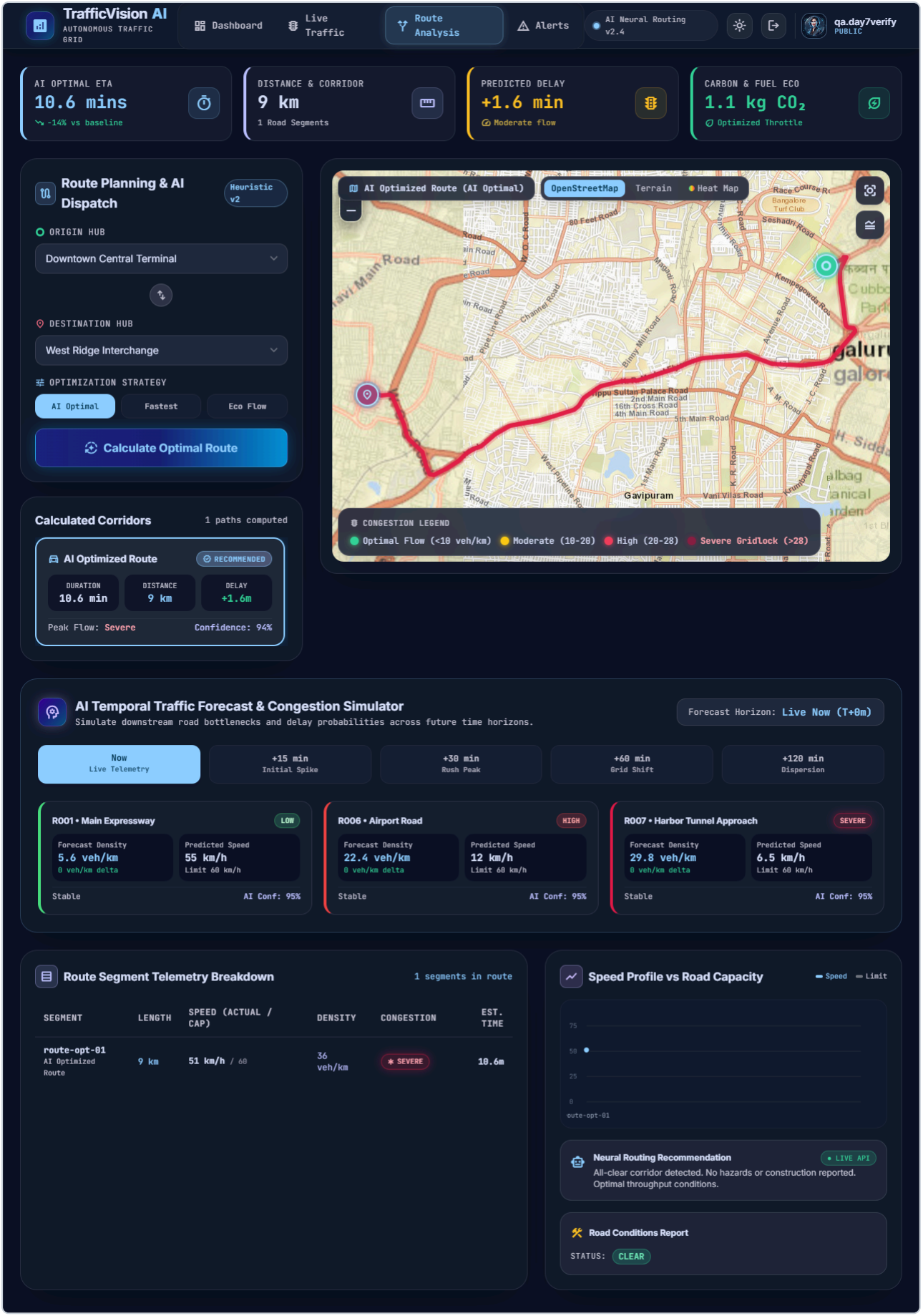
Proof

> **QA/milestone2/day5/milestone2_day5_integration_cycle.md** — full attempt-by-attempt record.

Attempt timeline

- **Attempt 1 (09-13):** blocked immediately — signup rejected (DEF-M2-016), login froze in an infinite redirect loop (DEF-M2-017), and even bypassing that, working backend calls were silently aborted and replaced with mock data (DEF-M2-018).
- **Attempt 2 (09-14):** frontend fixes landed for 016/017/018 (retry logic). Route recalculation (DEF-M2-007) confirmed fixed. But the app still silently showed mock data — found a new defect, DEF-M2-019 (a coordinate-order bug: hub coordinates are `[lat,lng]`, the backend's route geometry is GeoJSON `[lng,lat]`, and a stub-detection check compared them without swapping, always rejecting genuine data as fake). Separately, RBAC testing surfaced **DEF-M2-020**: the public registration endpoint accepted `role: "admin"` from anyone, with a real JWT to prove it — a genuine security hole, not a UI bug.
- **Attempt 3 (09-15):** backend redelivered. DEF-M2-020 and the historical data merge both confirmed fixed live (registering as "admin" now correctly returns a `PUBLIC`-role token; `/traffic/history`, `/aggregate`, `/road-condition` all return real 200 data). Frontend still had DEF-M2-019 open — badge still read "MOCK DATA" even with a fully working backend underneath.
- **Attempt 4 (09-15, same day):** frontend redelivered with the coordinate-order fix. Live-tested the complete flow — **the data-source badge read "LIVE API" for the first time this milestone**, with realistic decimal ETA/distance values that correctly updated on destination change.

The defining screenshot of this milestone



Attempt 4, 2026-09-15 — Route Analysis page after a real login and a destination change, hitting the real backend end-to-end. Data-source badge reads **"LIVE API"**, not mock — the first time this milestone real backend data reached the screen.

Still outstanding before Day 5 is fully closed

- Prediction Service completion (Intern 1) — full FR-02/03/04 scope can't be exercised until peak-hours/delay exist
- A dedicated RBAC pass across admin/operator/public against the now-secured role boundary (prior RBAC checks predate the DEF-M2-020 fix or ran against mock data)
- Confirming other flows (Live Traffic, dashboards) hold up under real data the same way Route Analysis just did

Day 6 — Code Quality & Regression Complete

SRS requirement	Regression-test all Milestone 1 features to confirm nothing broke; run static code-quality checks on new Milestone 2 code.
What was done	Both halves executed for real, with live evidence — not just re-stated as "done."

Proof — code quality

> **QA/milestone2/day6/milestone2_day6_code_quality_report.md** — Black, Flake8, ESLint, and Prettier all run against the real codebase (first fixing a config gap: no `.prettierignore` existed, so the format check was scanning the entire `.venv` before reaching real files). Key finding: 60% of all backend Flake8 violations (164 of 275) sit in dead, non-authoritative duplicate code, not the active codebase — recommended deletion over reformatting. Frontend ESLint issues were mostly false positives from a cross-file-globals pattern, not real bugs.

Proof — regression

> **QA/milestone2/day6/milestone2_day6_regression_report.md** — the first regression pass run against a genuinely live backend. Correctly reported a **FAIL** verdict at the time (Auth and Traffic were broken at the contract-path level by then-unfixed routing bugs) — exactly what a regression pass exists to catch. Also where the port-mismatch defect (DEF-M2-015) and the cold-start latency observation were first found.

Note: this regression pass has since been superseded by the live, passing results documented under Day 5 — kept here as the honest historical record of what regression testing found at the time, per the actual SRS Day 6 timing.

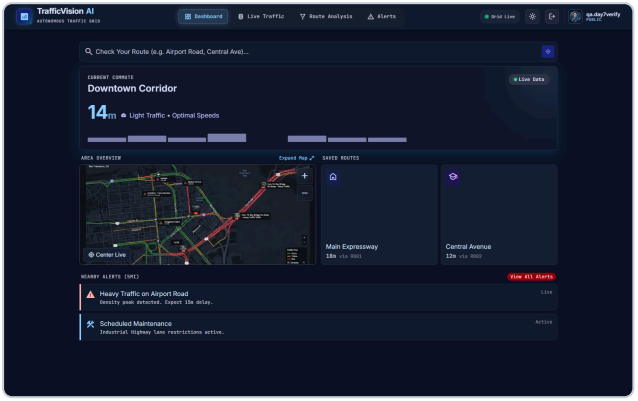
Day 7 — Final Re-verification & Ongoing Sign-off In progress

SRS requirement	Run the final Milestone 2 validation cycle (smoke + integration + performance sanity + RBAC); compile the QA sign-off report and test summary. No deployment activity.
What was done	Re-verification became an ongoing, repeated activity rather than a single final pass, since the team delivered fixes across three separate days (09-13, 09-14, 09-15) and each one was re-tested live the same day it landed. This document became the running log of that process.

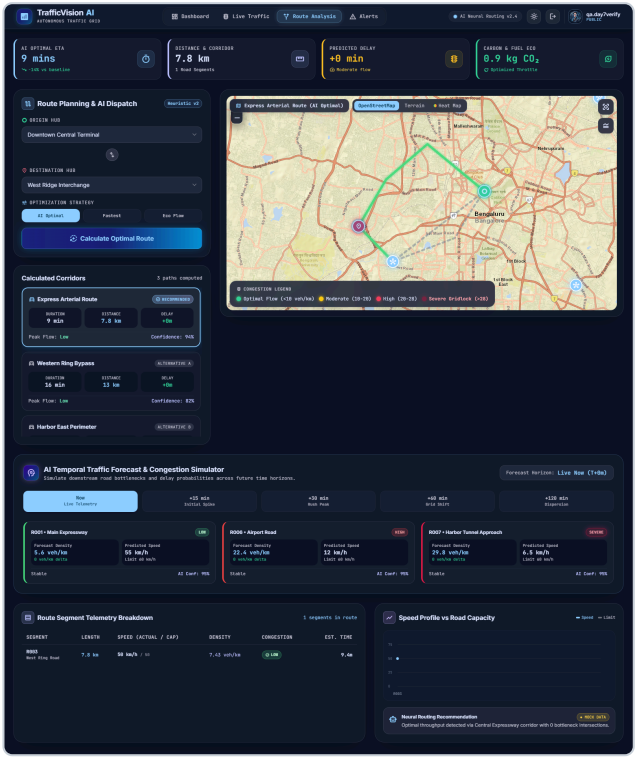
Proof

> **QA/milestone2/day7/milestone2_day4_5_7_reverification.md** — full re-verification log across all three redelivery rounds, including the discovery that a folder shared as "integration work" turned out to be a duplicate of an already-present file (confirmed via SHA-256 hash comparison), and the finding that the SRS's own numbering for Day 2/3's deliverables didn't match how the work was actually sequenced by the team — corrected and documented rather than silently worked around.

> **QA/milestone2/day7/screenshots/** and `day7_reverify_results.json` — supporting screenshots and raw automated-test output from the live re-verification runs.



Real login via the actual form (not a workaround) — clean redirect, no infinite loop (DEF-M2-017 re-verification).



Route Analysis after changing destination — KPIs and map genuinely recalculated (DEF-M2-007 re-verification).

What's left before final sign-off can be written

- Intern 1's Prediction Service completion (peak-hours, delay, contract resolution)

- The RBAC and full-app regression pass noted under Day 5
- A root `.gitignore` decision — added by QA and confirmed working (excludes all `.env` files with real credentials found across the backend and data folders), but no commit has been made yet this milestone, by design, pending team sign-off

Defect Summary — All 20, Current Status

Full detail, reproduction steps, and evidence for every item below live in `QA/milestone2/day1/defect_tracker.md`. This table is a condensed index.

ID	Feature	Severity	Status	Owner
DEF-M2-001	Backend repo structure (nested delivery, embedded .git)	**High**	**Fixed (QA)**	QA
DEF-M2-002	Auth API missing route prefix	**Critical**	**Fixed, verified live**	Backend
DEF-M2-003	Traffic Monitoring double-prefixed routes	**High**	**Fixed, verified live**	Backend
DEF-M2-004	Route Analysis double-prefixed routes	**High**	**Fixed, verified live**	Backend
DEF-M2-005	Prediction API contract mismatch (POST vs documented GET)	Medium	**Open**	AI/ML
DEF-M2-006	Dead/broken module, unused import	Low	**Open**	Backend
DEF-M2-007	Route Analysis page — stale data on control change	**High**	**Fixed, verified live**	Frontend
DEF-M2-008	Unpinned scikit-learn version (model reproducibility risk)	Low	**Open**	Backend/ML
DEF-M2-009	CI never ran backend's own test suite	Medium	**Fixed (QA)**	QA
DEF-M2-010	CI "verify backend import" step silently broken	Medium	**Fixed (QA)**	QA
DEF-M2-011	Flake8 violations concentrated in dead code	Low	**Open**	Backend
DEF-M2-012	ESLint false-positive on cross-file globals	Low	**Open**	Frontend
DEF-M2-013	Plaintext credential in Data module .env	Medium	**Open**	Data / whole team
DEF-M2-014	Dev Docker stack could never start (missing env file, wrong DB host)	**High**	**Fixed (QA)**	QA
DEF-M2-015	Frontend/backend port mismatch	**High**	**Fixed, verified live**	Frontend
DEF-M2-016	Signup role rejected by backend enum	**High**	**Fixed, verified live**	Frontend
DEF-M2-017	Login infinite redirect loop (role mismatch)	**Critical**	**Fixed, verified live**	Frontend
DEF-M2-018	apiFetch retry logic aborting working requests	**Critical**	**Fixed, verified live**	Frontend
DEF-M2-019	Route stub-detection: swapped lat/lng rejects real data	**Critical**	**Fixed, verified live**	Frontend

DEF-M2-020	Security — unauthenticated privilege escalation via self-registration	**Critical**	**Fixed, verified live**	Backend
DEF-M2-021	Prediction GET endpoints crash — wrong shape assumed from get_road_traffic()	**High**	**Open**	AI/ML
DEF-M2-022	get_road_condition() queries the wrong table (roads instead of road_conditions)	Medium	**Open**	Backend
DEF-M2-023	Database was never seeded — endpoints silently served fallback sample data	**High**	**Fixed (QA)**	QA

5	**9**	**5**	**4**
CRITICAL, ALL FIXED	HIGH, 8 FIXED	MEDIUM, 2 FIXED	LOW, ALL STILL OPEN

Remaining open items: 1 High (DEF-M2-021, Prediction Service crash, awaiting Intern 1's fix), the rest Low/Medium severity code-hygiene, dependency-pinning, and one query-target bug (DEF-M2-022) — nothing release-blocking remains open at Critical severity.